

```
#### Simple Linear Regression ####

# Lecture Slides

#####
#slide 14
#####

rm(list=ls()) #to clear the global environment

#read in data, either manually or from a csv file
xc <- c(2,3,4)
yc <-c(3,1,2)

plot(xc,yc)

#find r, the sample correlation coefficient
#find the means
xbar <- mean(xc)
ybar <- mean(yc)
#find the sums of squares
SSxy <- sum((xc-xbar)*(yc-ybar))
SSx <- sum((xc-xbar)^2)
SSy <- sum((yc-ybar)^2)
#caluclate r
s14_r <- (SSxy)/(sqrt(SSx*SSy))

round(s14_r, digits = 2) #round function rounds the answer to the desired
number of decimal places

#OR use R's built-in function
s14_cor <-cor(xc,yc)
s14_cor

#####
#slide 17
#####

rm(list=ls()) #to clear the global environment

#read in the data (either manually like here, or with a .csv file)
x_lectures <-
c(46,10,38,27,45,26,35,45,48,20,30,27,38,12,28,40,38,47,36,40)
y_marks <- c(80,20,59,34,71,55,50,78,81,28,50,47,77,18,41,79,68,88,66,70)

#plot the points
plot(x_lectures,y_marks)

#####

#find r, the sample correlation coefficient
#find the means
xbar <- mean(x_lectures)
```

```

ybar <- mean(y_marks)
#find the sums of squares
SSxy <- sum((x_lectures-xbar)*(y_marks-ybar))
SSx <- sum((x_lectures-xbar)^2)
SSy <- sum((y_marks-ybar)^2)
#caluclate r
s17_r <- (SSxy)/(sqrt(SSx*SSy))

round(s17_r, digits = 2) #round function rounds the answer to the desired
number of decimal places

#OR use R's manual function:
round(cor(x_lectures,y_marks), digits = 2)

#####
#slide 18 - 19
#####

#test the significance of the correlation coefficient:
#specify n, the number of observations
n <- length(x_lectures)
#manually calculate the test statistic
tstat <- (s17_r*sqrt(n-2))/(sqrt(1-s17_r^2)) #calculates the test statistic
tstat
#"manually" calculate the p-value
pval <- 2*pt(q=tstat, df = n-2, lower.tail = F) #calculate the p-value for
the test statistic (why do we say 2*pt() in this case?)
pval

#OR use R's manual function:
cor.test(x_lectures, y_marks)

#####
#slide 28
#####

#fitting the simple linear regression model:
fit <- lm(y_marks ~ x_lectures)
summary(fit)

#####
#slide 31
#####

#RSE calculated manually (alternatively read off the RSE from the output in
summary(fit)):
se_b1 <- 0.1418 #read from summary(fit)
rse <- se_b1*sqrt(SSx)
round(rse, digits = 2)

#####
#slide 32-33
#####

```

```

#test the significance of the betal estimate:
#manually calculate the test statistic
b1 <- 1.8250 #read off from summary(fit)
tstat_b1 <- b1/se_b1 #calculates the test statistic
tstat_b1
#"manually" calculate the p-value
pval_b1 <- 2*pt(q=tstat_b1, df = n-2, lower.tail = F) #calculate the p-
value for the test statistic (why do we say 2*pt() in this case?)
pval_b1
#OR just read it off from summary(fit)

#####
#slide 35
#####

#calculate confidence intervals for beta0 and betal "manually"
b0 <- -3.6851 #read from summary(fit)
se_b0 <- 5.0333 #read from summary(fit)
#limits from the formulas:
ci_b0_upper <- b0 - qt(p=0.025, df = n-2, lower.tail = T)*se_b0
ci_b0_lower <- b0 + qt(p=0.025, df = n-2, lower.tail = T)*se_b0
ci_b1_upper <- b1 - qt(p=0.025, df = n-2, lower.tail = T)*se_b1
ci_b1_lower <- b1 + qt(p=0.025, df = n-2, lower.tail = T)*se_b1
print(paste("Beta0 CI: (",ci_b0_lower, ci_b0_upper,")"))
print(paste("Beta1 CI: (",ci_b1_lower, ci_b1_upper,")"))
#OR with R's manual function:
confint(fit)

#####
#slide 37
#####

#"Manually" test for overall model significance (OR read it off from the
summary(fit) function)
#calculate tabled values
SSreg <- sum((fit$fitted.values-ybar)^2)
SSresid <-sum((y_marks-fit$fitted.values)^2)
SStot <- sum((y_marks-ybar)^2)
MSreg <- SSreg/1
MSresid <-SSresid/(n-2)
#calculate the F statistic
Fteststat <-MSreg/MSresid
round(Fteststat, digits = 2)
#calculate the p-value
pval_Fteststat <- pf(q = Fteststat, df1 = 1, df2 = n-2, lower.tail = F)
pval_Fteststat

#####
#slide 38
#####

```

```

#Manually calculate R2, the coefficient of determination (or read it off
from the summary(fit) model)
r2 <- SSreg/SStot
round(r2, digits = 4)

#####
#slide 41 - 45
#####

#assess the model assumptions

#check linearity of data
plot(x_lectures, y_marks)

#check normality of residuals
  #histogram of residuals
hist(fit$residuals)
  #qqplot of residuals
qqnorm(fit$residuals)
qqline(fit$residuals, col = "red")

#assess assumption of constant variance
plot(fit$fitted.values, fit$residuals)

#assess assumption of independence of errors
plot(x_lectures, fit$residuals)

#####
#slide 48
#####

#manual prediction of one value when x = 35 lectures
yhat <- b0 +b1*(35)
round(yhat, digits = 2)

#####
#slide 50
#####

#prediction intervals with R's built-in functions
  #predict for a single individual (with its PI)
xval <- 35
predict(fit, newdata = list(x_lectures = xval), interval = "prediction")
  #predict for lecture range (not including PI)
x_lectures_new <- data.frame(x_lectures = 10:48)
predict(fit, newdata = x_lectures_new)

#####

# Additional worked example SLR slides (Iris dataset)

#####
rm(list=ls()) #to clear the global environment

```

```
#access the Iris dataset (built into R)
x <- iris$Sepal.Length
y <- iris$Petal.Length
plot(x,y)

#calculate correlation coefficient
cor(x,y)

#fit the simple linear regression model
fit <- lm(y~x)
summary(fit)

#predict petal length and find confidence interval
xval <- 5.3
predict(fit, newdata = list(x = xval), interval = "confidence")

#assess the model assumptions
#check normality of residuals
#histogram of residuals
hist(fit$residuals)
#qqplot of residuals
qqnorm(fit$residuals)
qqline(fit$residuals, col = "red")

# _____ #
# _____ #
```